

UTDT

Tecnología Digital V

Trabajo Práctico 2
Logística centralizada de primera milla

Integrantes

Franco Arrieta

Gonzalo Benzaquen

Julieta Kovalivker

Motivación

El crecimiento del e-commerce en los últimos años ha llevado a marketplaces y empresas logísticas de Latinoamérica y otras regiones (por ejemplo, Asia) a buscar estrategias para reducir sus costos operativos. Para vendedores grandes, con un alto volumen de ventas diario, suele resultar conveniente planificar visitas periódicas para recolectar los paquetes directamente en sus negocios. Sin embargo, esta operación resulta costosa en términos logísticos cuando se trata de vendedores pequeños (en adelante, vendedores), que despachan solo unos pocos ítems por día (por ejemplo, menos de 10 o 15). En estos casos, son los propios vendedores quienes deben encargarse de trasladar sus paquetes hasta un punto de recepción de la red, comúnmente denominado primera milla.

Estos puntos de recepción, a los que nos referiremos como depósitos, funcionan como nodos de consolidación, simplificando la operatoria logística general. Una modalidad habitual consiste en que cada vendedor elija libremente a qué depósito llevar sus paquetes, pudiendo además cambiar de depósito sin previo aviso. Estudios previos muestran que esta modalidad genera una utilización ineficiente de la capacidad de almacenamiento de la red: cuando un vendedor elige un depósito sin capacidad disponible, debe recorrer otros posibles destinos hasta encontrar uno que pueda recibir sus paquetes. Esto se traduce en reclamos, costos adicionales y una mala experiencia para los vendedores.

Esta problemática motiva las siguientes preguntas de investigación:

- ¿Es suficiente la capacidad actual de la red de depósitos, o es necesario expandirla para poder dar respuesta a la demanda de todos los vendedores?
- ¿Es posible encontrar una asignación de vendedores a depósitos tal que cada uno deba recorrer una distancia razonable para entregar sus paquetes?
- ¿Es factible desarrollar una herramienta que permita experimentar con distintos escenarios y obtener soluciones de buena calidad en pocos minutos?

Definición del problema y modelado

El problema descrito anteriormente puede ser modelado mediante el Problema de Asignación Generalizada (GAP, Generalized Assignment Problem) que puede ser formulado de la siguiente forma. Sea $N = \{1, \dots, n\}$ el conjunto de vendedores y $M = \{1, \dots, m\}$ el conjunto de depósitos. Cada depósito $i \in M$ tiene una capacidad máxima de recepción c_i , medida en cantidad de unidades. Dado un vendedor $j \in N$ y un depósito $i \in M$, d_{ij} denota la demanda (también en cantidad de unidades) a utilizar y c_{ij} el costo incurrido si j es asignado a i . Con estas definiciones, podemos representar una solución como una colección de conjuntos $\Gamma_1, \dots, \Gamma_m \subseteq N$, con Γ_i el subconjunto de vendedores asignados al depósito $i \in M$. Luego, el GAP consiste en:

1. asignar cada $j \in N$ a exactamente un depósito $i \in M$, es decir, $\Gamma_i \cap \Gamma_k = \emptyset$ si $i \neq k$, $i, k \in M$
2. la capacidad de cada depósito no debe ser excedida, es decir, para $i \in M$

$$\sum_{j \in \Gamma_i} d_{ij} \leq c_i$$

3. minimizar el costo total de la asignación dada por

$$\sum_{i=1}^m \sum_{j \in \Gamma_i} c_{ij}.$$

A su vez, definimos:

$$C_{\max} = \max_{i \in M, j \in N} c_{ij}$$

como la máxima distancia posible a recorrer por un vendedor. En caso de tener una solución parcial, donde haya vendedores que no pueden ser asignados a algún depósito, se asumirá una penalización de $3 \times C_{\max}$ por cada uno de ellos.

Modelos propuestos

Para abordar el problema de logística centralizada de primera milla propusimos e implementamos diversos modelos con el objetivo de optimizar la asignación de los vendedores hacia los depósitos. A continuación se describen los modelos implementados.

Diseño de estructuras

Para manejar la lectura de instancias y el formato de las soluciones, diseñamos dos clases para simplificar las mismas mediante métodos y atributos.

- GAPInstance(string ruta)

Dada la ruta a un archivo de instancia lee y guarda las variables (depósitos, vendedores, costos y capacidades) como atributos.

- Asignacion(const GAPInstance instancia)

Dada una instancia inicializa una asignación posible. Provee métodos (modificadores y observadores) y operadores usados por los modelos propuestos (asignar/desasignar vendedores a depósitos, leer asignación actual, ver factibilidad de cambios en la asignación). Una instancia de asignación siempre se mantiene factible y mantiene un puntero constante a la instancia, de modo tal que nunca se modifican las variables originales.

Por otra parte, todos los modelos propuestos a continuación fueron implementados como métodos de una clase la cual, dada una instancia, resuelve el problema.

- Solver(const GAPInstance instancia)

Heurística de Varianzas

Como primera intuición, buscamos evitar asignar vendedores a depósitos muy costosos cuando disponen de alternativas más económicas. Para ello, consideramos los costos de asignar cada vendedor a cada depósito y calculamos la varianza de dichos costos. En particular, para un vendedor $j \in N$ cuyos costos hacia los m depósitos son $C_j = (c_{1j}, c_{2j}, \dots, c_{mj})$ calculamos la varianza como

$$\text{Var}(C_j) = \frac{1}{M-1} \sum_{i \in M} (c_{ij} - \bar{C}_j)^2$$

donde

$$\bar{C}_j = \frac{1}{M} \sum_{i \in M} c_{ij}$$

es el costo promedio del vendedor j entre todos los depósitos.

Interpretamos que un vendedor con una varianza alta es más "exigente" respecto de los depósitos a los que puede ser asignado, ya que la elección del depósito tiene un impacto significativo en el costo. En cambio, una varianza baja indica que el vendedor es relativamente indiferente a la asignación, dado que el costo se mantiene similar independientemente del depósito seleccionado.

Si bien este criterio no garantiza que las demandas se distribuyan de manera óptima, sí nos permite priorizar la asignación de los vendedores más sensibles a la asignación y, de esta forma, evitar decisiones que generen costos elevados de manera innecesaria.

```
herusiticaVarianzas(instancia) // O(M*N + NlogN)
```

```

asignacion ← new Asignacion(instancia) // O(M)
costos ← costosPorVendedor(instancia) // O(1)
varianzas ← calcularVarianzas(costos) // O(M*N)
ordenarVarianzas(varianzas) // O(N*logN)
asignados ← 0
while asignados < N do // O(N)
    siguiente ← varianzas[ N - asignados - 1]
    vendedor ← siguiente.vendedor
    deposito ← depositoMinFactible(vendedor, costos[vendedor]) // O(M)
    asignación.asignar(deposito, vendedor) // O(1)
    asignados ← asignados + 1;
end while
return asignación

```

```
depositoMinFactible(vendedor, costos) // O(M)
```

```

depositoMin ← NULL
for d ∈ M do // O(M)
    if hayLugar(d, vendedor) then // O(1)
        if depositoMin == NULL ∨ costos[depositoMin] > costos[d] then
            depositoMin ← d
        end if
    end if
end for
if depositoMin == NULL then return M
return depositoMin

```

Heurística de Depósitos

Esta heurística surgió como un intento de mejora de la anterior. En un primer acercamiento al problema, en lugar de seleccionar el vendedor en base a la varianza en sus costos, lo hacíamos en base a la varianza de sus costos por unidad demandada (ratio). Experimentando con este modelo (finalmente descartado) notamos que en varias instancias muchos depósitos quedaban sin vendedores asignados (y muchos vendedores sin depósito).

A pesar de que la implementación mediante ratios fue descartada, nos sirvió como base para plantear una segunda heurística. En lugar de asignar por cada vendedor su depósito más económico, asignar para cada depósito, su vendedor disponible más conveniente. Para ello, buscamos ir llenando de forma iterativa depósitos con el vendedor factible de menor costo posible. Iteramos en orden desde el depósito de menor capacidad pues, si quedan para asignar al final, podría suceder que las demandas de los vendedores restantes excedan las capacidades de los depósitos disponibles y terminen sin ser asignados, lo cual es potencialmente muy costoso.

```
heuristicaDepositos(instancia) //  $O(M \cdot \log M + N^2 \cdot M)$ 
```

```
asignacion ← new Asignacion(instancia) //  $O(M)$ 
costos ← costosPorDeposito(instancia) //  $O(1)$ 
depositos ← ordenarDepositosPorCapacidad(instancia) //  $O(M \cdot \log M)$ 
asignados ← 0; cambie ← true
while cambie ∧ asignados != N do //  $O(N)$ 
    cambie ← false
    for d ∈ depositos do //  $O(M)$ 
        v ← vendedorMinFactible(d, costos[d]) //  $O(N)$ 
        if v != NULL then // si encuentre un vendedor factible...
            asignacion.asignar(d, v) //  $O(1)$ 
            asignados ← asignados + 1
            cambie ← true
        end if
    end for
end while
return asignacion
```

```
vendedorMinFactible(depósito, costos) //  $O(N)$ 
```

```
vendedorMin ← NULL
for v ∈ N do //  $O(N)$ 
    if hayLugar(deposito, v) ∧ depositoDe(v) == NULL then //  $O(1)$ 
        if vendedorMin == NULL ∨ costos[vendedorMin] > costos[v] then
            vendedorMin ← v
        end if
    end if
end for
```

return vendedorMin

Heurística de Demandas

Para el tercer modelo decidimos tomar decisiones en base a la demanda en vez del costo. Por cada vendedor, tenemos en cuenta la demanda promedio de cada vendedor e intentamos asignarlo al depósito factible al que le demande menos. Queremos ver si minimizar la cantidad enviada a cada depósito de los vendedores con demandas en promedio muy grandes, genera más espacio en cada uno y más asignaciones factibles, evitando en lo posible, que queden vendedores sin depósito.

En particular, para un vendedor $j \in N$ cuyas demandas hacia los m depósitos son $D_j = \{d_{1j}, d_{2j}, \dots, d_{mj}\}$ calculamos el promedio como

$$\bar{D}_j = \frac{1}{M} \sum_{i \in M} d_{ji}$$

La heurística fue implementada de forma iterativa buscando, en orden, el vendedor con mayor demanda y buscamos su respectivo depósito factible al que demande menos.

heuristicaDemandas() // $O(N \cdot M + N \log N)$

```
asignacion ← new Asignacion(instancia) //  $O(M)$ 
demandas ← demandasPorVendedor(instancia) //  $O(1)$ 
promedios ← calcularPromedios(demandas) //  $O(N \cdot M)$ 
ordenarPromedios(promedios) //  $O(N \cdot \log N)$ 
asignados ← 0
while asignados < N do //  $O(N)$ 
    siguiente ← promedios[ N - asignados - 1]
    vendedor ← siguiente.vendedor
    deposito ← depositoMinFactible(vendedor, demandas[vendedor]) //  $O(M)$ 
    asignación.asignar(deposito, vendedor) //  $O(1)$ 
    asignados ← asignados + 1
end while
return asignación
```

Búsqueda local - Swap

Para mejorar los resultados obtenidos por las heurísticas constructivas, implementamos una búsqueda local basada en intercambios de vendedores entre depósitos. De esta manera, en cada iteración del algoritmo se evalúa, para cada vendedor, los posibles swaps con otros vendedores, donde solo se efectuará el cambio si es que el mismo respeta la condición de capacidad de ambos depósitos y genera un menor costo al existente. Con este operador se busca explorar mejor el vecindario y alcanzar un mínimo local mediante un acercamiento del tipo best-improvement.

BLswap(asignacion) // $O(N^2)$

```
c' ← costo; v1' ← NULL; v2' ← NULL
for v1 ∈ N do //  $O(N)$ 
    for v2 ∈ {v1 + 1, N} do //  $O(M)$ 
        if esFactibleSwap(v1, v2) then //  $O(1)$ 
            c_nuevo ← costoPostSwap(v1, v2) //  $O(1)$ 
            if c_nuevo < c' then c' ← c_nuevo; v1' ← v1; v2' ← v2 end if
        end if
    end for
end for
if c' < costo then swap(v1', v2') end if //  $O(N)$ 
```

Búsqueda local - 2Swap

De la misma forma que implementamos el swap, implementamos un 2swap que realiza lo mismo pero intercambia a dos vendedores con dos vendedores (en lugar de uno con uno). No especificamos el pseudocódigo pues es casi idéntico al swap.

Búsqueda local - Relocate

Por otro lado, realizamos búsqueda local mediante relocate, donde probamos “reubicar” a los vendedores en otros depósitos. Pensamos que podría generar soluciones con menores costos al ser utilizada luego de una heurística ya que, al evaluar cada vendedor reubicado en cada depósito, vamos de a poco encontrando pequeñas mejoras a la solución parcial que, al juntarlas, podrían sumar un costo significativamente menor al provisto por la heurística. Al igual que en el swap, no podemos garantizar que relocate encontrara la mejor solución pero alcanzará un mínimo local. En el caso de relocate destacamos que el operador es conectado, podríamos (teóricamente, aceptando soluciones que empeoran) alcanzar un mínimo global.

BLrelocate(asignacion) // $O(N*M)$

```
c_best ← costo; v_best ← NULL; d_best ← NULL
for v ∈ N do //  $O(N)$ 
    d_actual ← depositoDe(v) //  $O(1)$ 
    for d ∈ M do //  $O(M)$ 
        if d == d_actual then continue end if
        if capacidadRemanente(d) > demandaDe(d, v) then //  $O(1)$ 
            c' ← costo - costoDe(d_actual, v) + costoDe(d, v) //  $O(1)$ 
            if c' < c_best then c_best ← c'; v_best ← v; d_best ← d end if
        end if
    end for
end for
if c_best < costo then relocate(d_best, v_best) end if //  $O(N)$ 
```

Metaheurística

Para optimizar las soluciones generadas por las heurísticas constructivas desarrollamos una metaheurística. Su objetivo es explotar al máximo el potencial de las búsquedas locales, combinándolas de distintas maneras para mejorar la calidad de las soluciones obtenidas.

En nuestra implementación aplicamos ILS (Iterated Local Search) ya que, al contar con tres heurísticas constructivas capaces de generar soluciones diversas, nos resultó interesante analizar si esta variabilidad en las soluciones iniciales conducía a mínimos locales diferentes. A su vez, dadas dos soluciones factibles del problema, es muy barato computacionalmente determinar qué solución es mejor (simplemente mirando costos).

Para intentar escapar de los mínimos locales, perturbamos la solución. Con perturbación nos referimos a “desasignar” a vendedores de sus respectivos depósitos para hacer luego una nueva asignación. Para ello, la metaheurística recibe un hiper parámetro k que indica el porcentaje de vendedores a perturbar (utilizamos un porcentaje para generar resultados proporcionales entre instancias de distintos tamaños). Por otra parte, tomamos otro hiper parámetro $iteraciones$ que define cuántas veces ejecutar ILS con una perturbación.

No está claro cómo fijar los hiper parámetros ni tampoco en qué orden ejecutar las operaciones de búsqueda local en ILS. Por eso tuvimos que probar diversas combinaciones entre ellas y comparar los resultados obtenidos. Las decisiones tomadas sobre el orden y frecuencia de las búsquedas locales y los valores de los hiper parámetros serán discutidas en el apartado [experimentación y discusión](#).

Metaheuristica(asignacion, k , iteraciones) // $O(iteraciones * N * (M + N))$

$kv = (asignacion.cantidadDeVendedores() * k) / 100$ // $O(1)$

actual $\leftarrow$ asignacion; best $\leftarrow$ asignacion // $O(N * M)$

for $i \in iteraciones$ **do** // $O(iteraciones)$

 LOCAL_SEARCH(actual) // $O(N * M + N^2)$

if actual.costo < best.costo **then** best $\leftarrow$ actual **end if** // $O(N * M)$

 actual $\leftarrow$ PERTURBAR_COPIAR(best, kv) // $O(N * kv + N * M) \rightarrow O(N^2 + N * M)$

end for

asignacion $\leftarrow$ best // $O(N * M)$

PERTURBAR_COPIAR(asignacion, k) // $O(N * K + N * M)$, con $k = \#vendedores$ perturbados

copia $\leftarrow$ asignación // $O(N * M)$

for $i \in k$ **do** // $O(K)$

$v_a_sacar \leftarrow vendedorAlteatorio()$ // $O(1)$

 copia.desasignar(v_a_sacar) // $O(N)$

end for

return copia

LOCAL_SEARCH(asignacion) // $O(N * M + N^2)$

agotado $\leftarrow$ false; max_iteraciones $\leftarrow$ 100 // arbitrario

for $j \in max_iteraciones \wedge !agotado$ **do** // $O(1)$


```

c ← asignacion.costo
BL2swap(asignacion) //  $O(N^2)$ 
BLswap(asignacion) //  $O(N^2)$ 
BLrelocate(asignacion) //  $O(N \cdot M)$ 
if asignacion.costo == c then agotado ← true end if
end for

```

Experimentación y discusión

Para este apartado realizamos una serie de experimentos sobre nuestros modelos. Para ello hicimos uso de las instancias provistas para benchmark encontradas en el siguiente [link](#). Para generar varios de los outputs y armar los gráficos usamos la herramienta de inteligencia artificial Claude. No obstante, todo el código generado fue corroborado y auditado por nosotros. Adjuntamos un link a la conversación con el modelo en cada caso según corresponda. Por otra parte, en el archivo README.md en la carpeta junto a los archivos se encuentran instrucciones sobre cómo ejecutar el código (también se encuentra en la página principal en [github](#)).

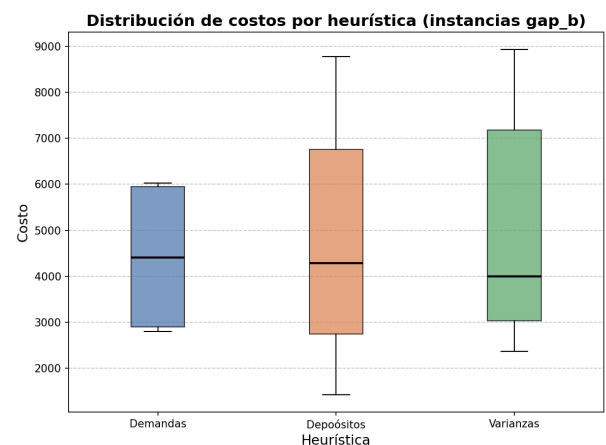
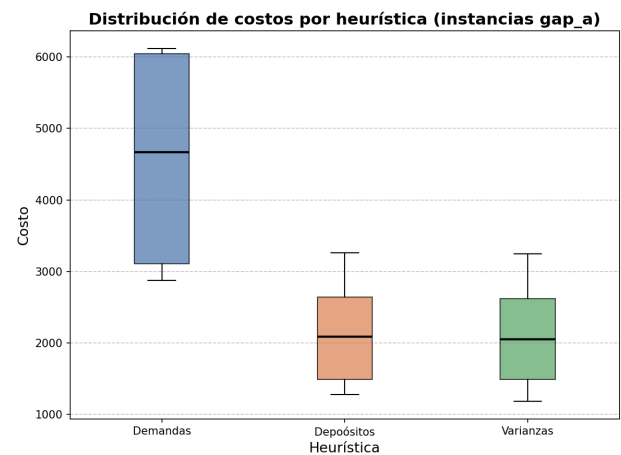
Comparación entre heurísticas por tamaño de instancia

Para comenzar a experimentar, quisimos averiguar cómo se comportan nuestras heurísticas. En términos generales, es difícil hablar sobre que es una “buena heurística”, nuestro acercamiento al problema fue computar las heurísticas sobre las instancias previstas y ver, en comparación, cuál da mejores resultados. Notamos al hacer esto que, al ser tan distintas las instancias de benchmark en términos del tamaño de N y M , cada heurística no funcionaba igual de bien para cada grupo. Es por eso que decidimos comparar las heurísticas separando por instancias.

Para las instancias a (tamaño pequeño), vemos que los costos son significativamente más altos con la heurística demandas en comparación a la de varianzas y depósitos. Por otra parte, no encontramos diferencia significativa entre los resultados de varianzas y depósitos.

Para las instancias b (tamaño mediano), vemos que los costos dan, en promedio, similares entre las tres heurísticas pero lo que sí difiere es la dispersión. Mientras que la heurística varianzas y de depósitos presentan bastante, la heurística demandas genera soluciones con costos más similares entre sí con menos outliers.

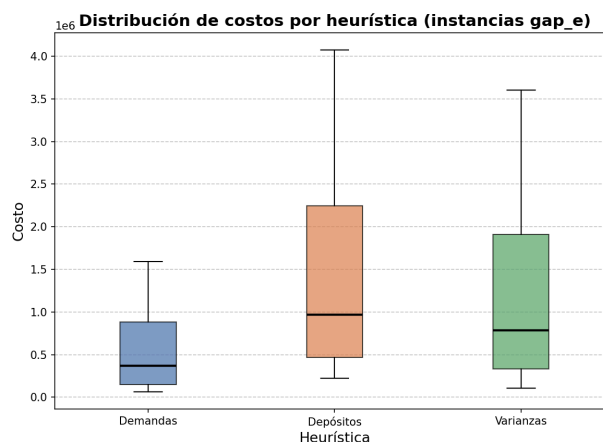
Para las instancias c (tamaño grande), no solo obtuvimos mucha menos dispersión en la heurística demandas, en comparación con varianzas y depósitos (como en las



instancias *b*), sino que en promedio, obtenemos costos significativamente menores.

En base a nuestras observaciones, nuestra hipótesis es la siguiente:

- En instancias pequeñas, las heurísticas, varianzas y depósitos brindan una mejor solución inicial.
- En instancias medianas, conviene utilizar la heurística de demandas si se busca mayor estabilidad en los resultados. Su menor dispersión indica que los costos obtenidos son más consistentes. Sin embargo, las diferencias entre las tres heurísticas son relativamente pequeñas.
- En instancias grandes, la heurística demandas brinda una mejor solución inicial.



Nos pareció particularmente interesante cómo cambia el funcionamiento de la heurística de demandas en función al tamaño de la instancia.

[Link](#) a la conversación con Claude.

Tuneo de parámetros

Además de comparar el rendimiento de los modelos heurísticos desarrollados, también nos interesa ver el comportamiento de las búsquedas locales y metaheurísticas realizadas. En particular, queremos ver como el tuneo de parámetros puede generar mejores o peores soluciones.

Tuneo del parámetro “k”

La metaheurística implementada realiza perturbaciones y optimiza mediante ILS por cada iteración. A la hora de perturbar la asignación, es importante tener en cuenta cuanto se perturba, es decir, cuántos vendedores serán “desasignados”. Para intentar encontrar un buen valor de *k*, comparamos los costos obtenidos por cada heurística luego de la optimización con distintos valores de *k* para todas las instancias de benchmark.

Para comenzar, con menos instancias y a ojo, probamos valores espaciados de *k*. En base a estas pruebas decidimos tomar: $k \in \{0, 5, 10, 20, 50\}$. Para el experimento, fijamos el hiper parámetro *iteraciones* en 10 de modo que sea consistente entre los modelos y posible de correr en tiempo razonable.

Para este experimento, nos interesa identificar, para cada instancia, cuál fue el valor de *k* que produjo la mejor solución. Es decir, entre todos los valores evaluados, se considera ganador aquel que alcanza el menor costo (en caso de empates, el punto se divide entre los ganadores).

Una vez determinado el valor ganador para cada instancia calculamos, para cada *k*, la diferencia porcentual promedio entre el costo de la solución que obtiene y el costo alcanzado por el ganador. De esta manera podemos cuantificar cuánto se aleja cada asignación del mejor resultado observado.

Heurística Varianzas

Analizando los resultados obtenidos podemos ver que con $k = 5\%$ generamos las soluciones con menores costos. A su vez, podemos remarcar $k = 10\%$ el cuál, a pesar de no destacarse en cantidad de victorias, obtuvo el gap (%) más bajo. Esto nos indica que, aunque gana con poca frecuencia, en promedio tiende a generar soluciones de buena calidad y relativamente cercanas a las óptimas. Esto podría indicar que valores de k entre 5%-10% generan buenas soluciones para esta heurística.

Heurística Depósitos

Podemos ver que con $k = 5\%$ y $k=20\%$ generamos las soluciones con menores costos, siendo los valores que obtuvieron la mayor cantidad de victorias. A su vez, el gap (%) entre estas es bajo, lo que indica que en los casos que no ganaban, generaban soluciones de calidad similar. En el otro extremo, $k = 0\%$ (es decir, sin perturbación) fue el peor caso. Obtuvo la menor cantidad de victorias y presentó el gap (%) más alto de toda la tabla ($\sim 7,09$). Esto evidencia la importancia de introducir cierto grado de perturbación en la búsqueda, ya que sin ella la heurística queda atrapada en mínimos locales de peor calidad.

Heurística Demandas

Finalmente analizando los resultados obtenidos con la heurística demandas, podemos ver aquí también que con $k = 5\%$ generamos las soluciones con menores costos, teniendo bastante gap (%) con los otros valores de k .

Este experimento muestra que valores de k entre 5% y 10% ofrecen mejores resultados para nuestras heurísticas. Como mejora, sería interesante explorar ese rango en mayor detalle para precisar un valor más óptimo.

A la hora de ejecutar este experimento, nuestro mayor problema fue el tiempo de cómputo. Debido a la complejidad de los operadores de búsqueda local correr muchas iteraciones por cada k nos resultaba inviable. Una posible mejora a este experimento sería aumentar el hiper parámetro *iteraciones*.

[Link](#) a la conversación con Claude.

Orden del VND

Al ejecutar ILS es importante definir el orden de los operadores de búsqueda local con el cual se realizará el VND (Variable Neighborhood Descent). Nos interesa encontrar un “buen orden” y frecuencia para la ejecución de los operadores (en el contexto de este trabajo, relocate, swap y 2swap). Probar todas las combinaciones de órdenes y cantidades de cada operador sería imposible ya que son infinitas. Es por eso

Heurística Varianzas		
k (%)	# victorias	gap (%)
0	6,166667	10,966205
5	11,5	0
10	3,5	3,224952
20	4	11,148095
50	0,666667	14,181729
100	1,166667	14,135477

Heurística Depósitos		
k (%)	# victorias	gap (%)
0	1,4	7,091333
5	8,733333	0
10	5,4	0,384079
20	8,733333	1,815934
50	2,733333	6,724767

Heurística Demandas		
k (%)	# victorias	gap (%)
0	6,2	4,922899
5	9,7	0
10	3,2	3,705292
20	5,2	9,530161
50	2,7	9,765532

que redujimos el problema a un máximo de tres operadores por iteración de ILS y entre las combinaciones posibles elegimos:

1. swap $\rightarrow$ relocate $\rightarrow$ 2swap
2. relocate $\rightarrow$ swap $\rightarrow$ relocate
3. *relocate* $\rightarrow$ 2*swap* $\rightarrow$ *relocate*
4. swap $\rightarrow$ relocate $\rightarrow$ swap
5. 2swap $\rightarrow$ relocate $\rightarrow$ swap
6. 2swap $\rightarrow$ swap $\rightarrow$ relocate

Computamos con cada heurística, todas las instancias y luego la metaheurística con cada orden del VND. Para cada combinación, guardamos el costo / vendedor de la solución y obtuvimos los datos mostrados en la tabla. El menor costo lo generó la combinación 3: *relocate* $\rightarrow$ 2*swap* $\rightarrow$ *relocate*. Resulta interesante notar que las combinaciones que incluyen el operador 2*swap* obtuvieron resultados consistentemente más bajos, lo que podría indicar que se trata de un buen operador para explorar el espacio de soluciones.

Órdenes del VND	
# Combinación	Costo / Vendedor
1	714.375
2	892.543
3	704.37
4	897.582
5	721.565
6	706.23

Tunéo del parámetro “iteraciones”

Otro experimento interesante consiste en evaluar distintos valores para el parámetro iteraciones y analizar su impacto sobre la calidad de las soluciones. Esto permitiría estudiar la relación entre el tiempo de cómputo invertido y la mejora obtenida en los resultados. La evaluación de este parámetro queda pendiente como una posible extensión del trabajo realizado.

Dificultades encontradas

Durante el desarrollo del trabajo nos encontramos con una serie de dificultades. En primer lugar, nos costó encontrar una forma objetiva de evaluar si nuestros modelos eran buenos. Al no contar con una cota inferior conocida para las instancias de benchmark, no teníamos una referencia clara contra la cual comparar nuestros resultados, por lo que tuvimos que conformarnos con comparaciones relativas entre nuestras propias heurísticas.

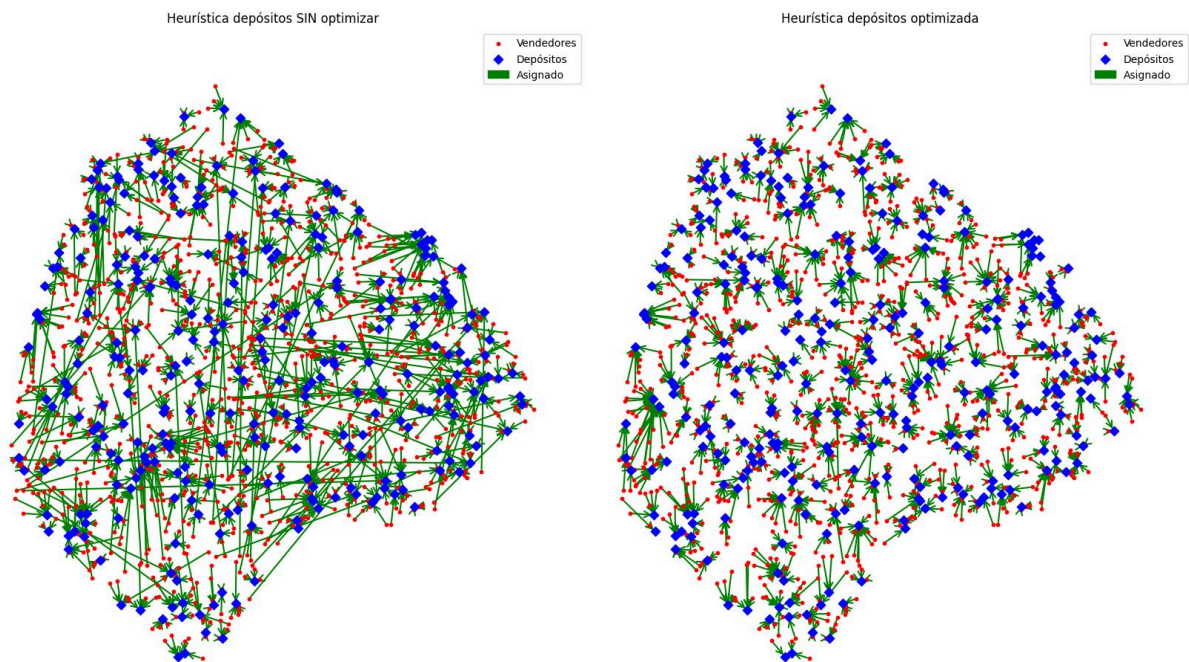
Por otra parte, al momento de tunear los hiper parámetros (k, iteraciones y orden del VND) no sabíamos en qué orden fijarlos, ya que estos no son independientes entre sí: el valor óptimo de k podría depender del orden del VND elegido, y viceversa. Al no poder explorar todas las combinaciones posibles, optamos por fijar los parámetros de a uno, asumiendo cierta independencia entre ellos, lo cual reconocemos como una simplificación que podría no reflejar el comportamiento óptimo conjunto de la metaheurística.

Finalmente, también tuvimos problemas para mantener los invariantes de nuestras estructuras de datos a medida que las heurísticas las modificaban incrementalmente. Cada movimiento entre vecindades tiene que mantenerse consistente en todas las estructuras auxiliares usadas para acelerar el cálculo de la función objetivo, y un desajuste entre ellas no producía errores evidentes sino soluciones subóptimas o inconsistentes, difíciles de detectar a simple vista. Esto nos obligó a destinar tiempo adicional a validar, mediante chequeos manuales, que el estado de las estructuras se mantuviera coherente luego de cada operación.

Análisis de caso real

En el marco del trabajo, se propuso analizar el desempeño de nuestro modelo sobre una instancia real. Para ello, ejecutamos las tres heurísticas propuestas y aplicamos la optimización con los parámetros ajustados a partir de nuestra experimentación previa. El mejor costo obtenido fue de 692, alcanzado luego de aproximadamente 2:20 minutos de ejecución. Esta solución surge de aplicar la heurística de depósitos con un valor de $k=5\%$ con *iteraciones* = 25 y orden de VND: 2swap $\rightarrow$ swap $\rightarrow$ relocate.

Dicha asignación se ve representada en la siguiente visualización. A la izquierda, la asignación obtenida tras ejecutar solo la heurística (costo inicial de 1378.6). A la derecha, la asignación luego de optimizar (costo final de 692).



Conclusión

A lo largo de este trabajo abordamos el problema de logística centralizada de primera milla, e implementamos tres heurísticas constructivas cuyo rendimiento resultó depender del tamaño de la instancia. Sobre estas soluciones iniciales aplicamos búsquedas locales y una metaheurística ILS con perturbación, cuyo tuneo de hiper parámetros mostró que el impacto de k y el orden del VND varía según la heurística utilizada. Finalmente, la aplicación sobre una instancia real confirmó que la herramienta desarrollada permite obtener asignaciones de buena calidad en tiempos razonables, respondiendo así a las preguntas de investigación planteadas al comienzo del trabajo.